

PROJECT REPORT

SCOPE

Space Collision & Orbital Proximity Evaluation System

Submitted for the course

204ES: Object Oriented Programming

Submitted by	: Dhruvansh Panchal
ID	: 25EC417
Guide / Faculty	: Mayur Sevak
Institution	: Birla Vishvakarma Mahavidyalaya
Academic Year	: 2026-27

Declaration

I hereby declare that the project report entitled **"SCOPE: Space Collision & Orbital Proximity Evaluation System"** has been prepared and submitted by me as part of the academic requirements for the course **204ES – Object Oriented Programming**. This project represents my individual effort to apply the theoretical concepts and practical programming techniques learned during the course to the development of a simulation-based software application.

I declare that the project has been developed using standard C++17 as a console-based application, with the primary objective of demonstrating the application of object-oriented programming principles in a simplified space-object proximity and collision-risk evaluation environment.

I further declare that the orbital data, object parameters, simulated movements, closest-approach calculations, relative velocity values, and risk classifications used in this project are intended solely for educational and demonstration purposes. The orbital model is a simplified two-dimensional circular model, and the risk-assessment mechanism is based on project-defined scoring criteria and thresholds. The system is not intended to represent an operational space surveillance platform or a professionally validated collision-prediction system.

I confirm that the information, concepts, and external materials referred to during the preparation and development of this project have been acknowledged appropriately in the references section of this report. To the best of my knowledge, this report does not knowingly contain plagiarized material or misrepresented results. Any tools, resources, or external references used in accordance with the requirements of the project have been utilized for academic and educational purposes.

I understand that the project report is being submitted for academic evaluation, and I accept responsibility for the authenticity, accuracy, and integrity of the work presented herein. I further acknowledge that any violation of the academic integrity guidelines or institutional requirements may be subject to the applicable rules and regulations of the institution.

Therefore, I hereby affirm that this declaration is true to the best of my knowledge and beliefs.

Date: _____

Signature: _____

Dhruvansh Panchal

Acknowledgement

I would like to express my sincere gratitude to everyone who has supported and guided me throughout the successful completion of this project.

First and foremost, I would like to thank my project guide and mentor Prof. Mayur Sevak for their valuable guidance, continuous support, encouragement, and constructive suggestions throughout the development of this project. Their knowledge, experience, and guidance helped me gain a deeper understanding of the subject and overcome the challenges encountered during the project.

I would also like to express my heartfelt gratitude to the Head of the Department Dr. Bhargav Goradiya and all faculty members for providing me with the necessary knowledge, resources, and encouragement to complete this project successfully. Their valuable insights and academic support played an important role in enhancing my understanding and improving the quality of my work.

I am thankful to my institution and department for providing a supportive learning environment, the necessary facilities, and the opportunity to work on this project. This experience has helped me strengthen my practical knowledge, develop problem-solving skills, and gain valuable experience in applying theoretical concepts to practical situations.

I would also like to extend my sincere thanks to my friends and classmates for their cooperation, helpful discussions, suggestions, and encouragement throughout the project. Their support and willingness to share ideas made the entire process more productive and enjoyable.

Finally, I would like to express my deepest gratitude to my parents and family members for their constant love, patience, motivation, and unwavering support. Their encouragement has always inspired me to work hard, remain focused, and strive for excellence.

This project has been a valuable learning experience that has contributed to my academic and personal development. I am sincerely grateful to everyone who directly or indirectly contributed to the successful completion of this project.

Thank you all for your valuable support and encouragement.

Dhruvansh Panchal

Abstract

The rapid growth of space exploration, satellite deployment, and other space-related activities has led to a significant increase in the number of artificial objects orbiting the Earth. These objects include operational satellites, inactive satellites, discarded rocket bodies, and fragments of space debris. The increasing density of objects in Earth's orbit raises concerns about orbital congestion, close approaches, and the potential risk of collisions. Such collisions can damage operational spacecraft, interrupt essential satellite-based services, and generate additional debris, further increasing the complexity of the space environment. Therefore, evaluating the proximity and relative motion of orbital objects is an important aspect of understanding space collision risks.

The **Space Collision & Orbital Proximity Evaluation System (SCOPE)** is a simplified educational software application developed using standard C++17 to demonstrate the fundamental concepts involved in orbital proximity analysis and collision-risk assessment. The project provides a console-based simulation environment in which satellites, space debris, and rocket bodies are represented as individual objects with their own attributes and orbital parameters. The system follows an object-oriented design approach, using an abstract base class and derived classes to model different categories of space objects while maintaining a structured, modular, and extensible program architecture.

In SCOPE, each space object follows a simplified two-dimensional circular orbital model. Based on the defined orbital parameters, the application simulates the movement of objects over a specified period and evaluates their relative positions at successive simulation steps. The system examines every possible pair of objects to identify the minimum separation distance, or closest approach, during the simulated interval. In addition to determining the closest approach, it calculates the relative velocity between the objects to provide further information about the nature of each encounter. These calculations form the basis for evaluating the proximity of objects and estimating their encounter risk within the assumptions of the simulation model.

To make the results easier to interpret, SCOPE implements a transparent, project-defined risk-scoring mechanism. Based on the calculated encounter parameters and predefined thresholds, each encounter is classified into one of four risk levels: LOW, MODERATE, HIGH, or CRITICAL. This classification provides a clear and systematic way to compare encounters and identify potentially concerning close approaches within the simulated environment. The scoring model is intended for educational demonstration and does not represent a validated operational collision-probability model.

The application also includes data persistence and reporting capabilities. Users can save space-object data to text files and load previously stored data for subsequent simulations. Following the evaluation process, the system can generate a risk report containing relevant encounter information, including the objects involved, their closest approach distance, relative velocity, and assigned risk classification. The report can also be saved to a file for future reference, analysis, and documentation. Exception handling and file-handling mechanisms are incorporated to improve the program's robustness and manage potential errors during data processing and file operations.

A major objective of the project is to demonstrate the practical application of object-oriented programming principles and modern C++ programming techniques. The implementation incorporates classes and objects, encapsulation, inheritance,

abstraction, runtime polymorphism, function overloading, operator overloading, static members, templates, Standard Template Library (STL) containers, exception handling, and file handling. The project consists of 12 classes and approximately 1,500 lines of code. Its functionality has been evaluated using 14 automated test cases, all of which pass, providing evidence that the implemented features behave as expected under the tested conditions.

In conclusion, SCOPE demonstrates how fundamental C++ programming concepts can be combined to develop a structured simulation and risk-evaluation application inspired by the challenges of orbital congestion and space debris management. It provides an educational foundation for understanding object modelling, orbital motion simulation, pairwise proximity calculations, relative velocity analysis, risk classification, and report generation. However, the system uses simplified two-dimensional circular orbits and project-defined risk thresholds. It does not incorporate the precision tracking data, full orbital dynamics, uncertainty modelling, or validated collision-probability calculations required by professional conjunction assessment systems. Consequently, SCOPE is intended strictly as an educational model and is not suitable for operational collision prediction or real-world spacecraft safety decisions.

Table of Contents

1. Introduction 5
2. System Requirements and Tools 6
3. Theoretical Background 6
4. System Design 8
5. OOP Concepts Used 11
6. Mathematical Model 13
7. Algorithms 14
8. Implementation 15
9. User Interface and Sample Output 20
10. Testing 24
11. Results and Discussion 26
12. Limitations 26
13. Future Scope 27
14. Conclusion 27
15. References 27
Appendix A: Compiling and Running 28

List of Figures

Figure 4.1: System architecture of SCOPE 8
Figure 4.2: UML-style class diagram of the object hierarchy 8
Figure 4.3: Project workflow 9
Figure 9.1: Separation between the three flagged pairs 23

1. Introduction

1.1 Background

Earth's orbit is shared by active satellites, spent rocket stages and many fragments of broken objects, collectively called space debris. Objects in low Earth orbit travel at roughly 7 to 8 kilometres per second, so a collision between two objects, even a small fragment and a satellite, can be destructive. A collision also creates more fragments, which increases the risk for other spacecraft.

When two objects pass close to each other the event is called a **conjunction**. Space agencies monitor these events with precise tracking data, estimate the miss distance and the probability of collision, and sometimes move a satellite to avoid a collision. SCOPE is a small educational model of the first part of that process: finding how close two objects come and judging how serious the encounter is.

1.2 Problem Statement

The increasing number of satellites and space debris in Earth's orbit raises the risk of close approaches and potential collisions. Continuous monitoring of orbital objects is essential to identify hazardous encounters and maintain safe space operations. Early risk assessment can help prevent collisions and protect valuable space assets.

1.3 Motivation

The problem naturally involves several related kinds of objects that share some properties (identity, orbit) but behave differently (a satellite can manoeuvre; debris cannot). This is exactly the situation that Object Oriented Programming is designed for. The project lets a second-year student apply OOP to a meaningful, space-related problem using simple mathematics.

1.4 Objectives

1. Design a clean class hierarchy for satellites, space debris and rocket bodies using abstraction, inheritance and polymorphism.
2. Simulate simplified circular orbits using basic physics equations.
3. Detect the closest approach of every pair of objects, including the time and the relative velocity.
4. Classify the risk of each encounter using a transparent, explainable scoring model.
5. Provide persistent storage (data files) and risk reports using file handling.
6. Handle invalid input and unusual situations with exception handling so that the program does not crash.

1.5 Scope of the Project

Included: 2D circular orbits, simulated data, closest-approach detection for every pair, project-defined risk levels, a console menu, text-file storage and report generation.

Not included: real tracking data, three-dimensional or elliptical orbits, orbital perturbations, measurement uncertainty, probability-of-collision calculation, avoidance-manoeuvre planning, or a graphical interface.

1.6 Important Disclaimer

SCOPE is an **educational simulation**. It uses simulated data and a simplified model. It is **not** an operational collision-prediction system. The risk thresholds are defined by this project for teaching purposes and are **not** official standards of ISRO, NASA, ESA or any other organisation.

2. System Requirements and Tools

2.1 Hardware Requirements

- Any personal computer or laptop that can run a modern operating system.
- About 100 MB of free disk space (the program itself is very small).

2.2 Software Requirements

Item	Requirement
Operating system	Windows, Linux or macOS
Compiler	g++ with C++17 support (version 9 or newer); MinGW-w64 or MSYS2 on Windows
Editor	Visual Studio Code (or any text editor)
Build tool (optional)	GNU Make
Libraries	None. Only the C++ standard library is used
Internet / APIs / databases	Not required

2.3 Technologies Used

Technology	Use in SCOPE
C++17	Core language: classes, templates, smart pointers, exceptions
STL	`vector`, `map`, `unique_ptr`, `sort`, `any_of`
File handling	`ifstream`, `ofstream`, `std::filesystem` for data and report files
g++ / MinGW	Compiling the project
Make	Simple build and test commands
Git and GitHub	Version control and project submission

3. Theoretical Background

3.1 Orbital Terms

Term	Meaning
Earth radius (R)	Mean radius of Earth, 6371 km
Altitude (h)	Height of the object above Earth's surface
Orbital radius (r)	Distance from Earth's centre: r = R + h
Circular orbital speed (v)	Speed needed to stay in a circular orbit at radius r
Angular velocity (ω)	How fast the position angle changes, in radians per second
Orbital period (T)	Time for one complete revolution
Prograde / retrograde	Moving in the same direction as Earth's rotation / the opposite direction
Conjunction	Two objects passing close to each other
Closest approach	The smallest separation between two objects, and the time at which it occurs
Relative velocity	Speed of one object as seen from the other

4. System Design

4.1 Architecture

SCOPE has a layered design. The user interface (`ConsoleUI`) only coordinates; it contains no physics. Four manager classes do the work: `Simulation` owns the objects and performs the orbital calculations, `RiskAnalyzer` evaluates risk, `DataManager` handles data files and `ReportGenerator` creates reports. The physical objects form a class hierarchy with `OrbitalObject` at the top.

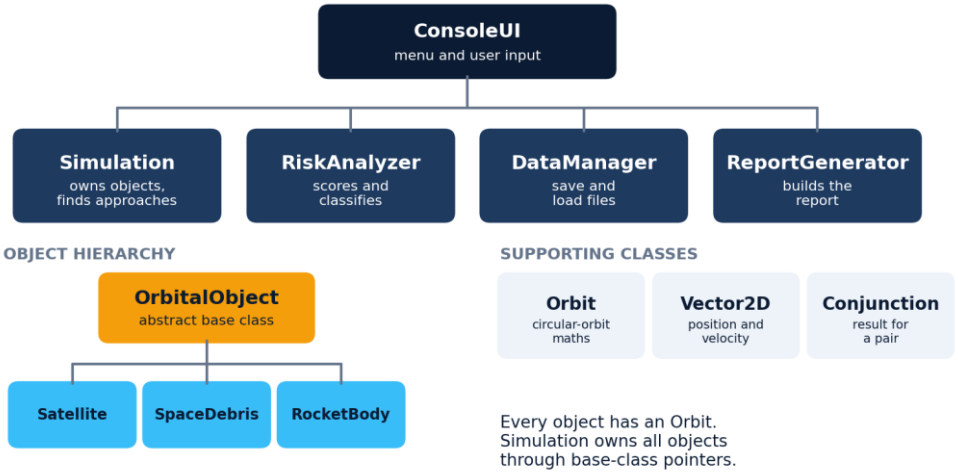


Figure 4.1: System architecture of SCOPE

4.2 Class Diagram

Figure 4.2 shows the class hierarchy in UML style. The three concrete classes inherit from the abstract base class `OrbitalObject`. Every object contains an `Orbit` (composition), and `Simulation` owns all objects through `unique_ptr`.

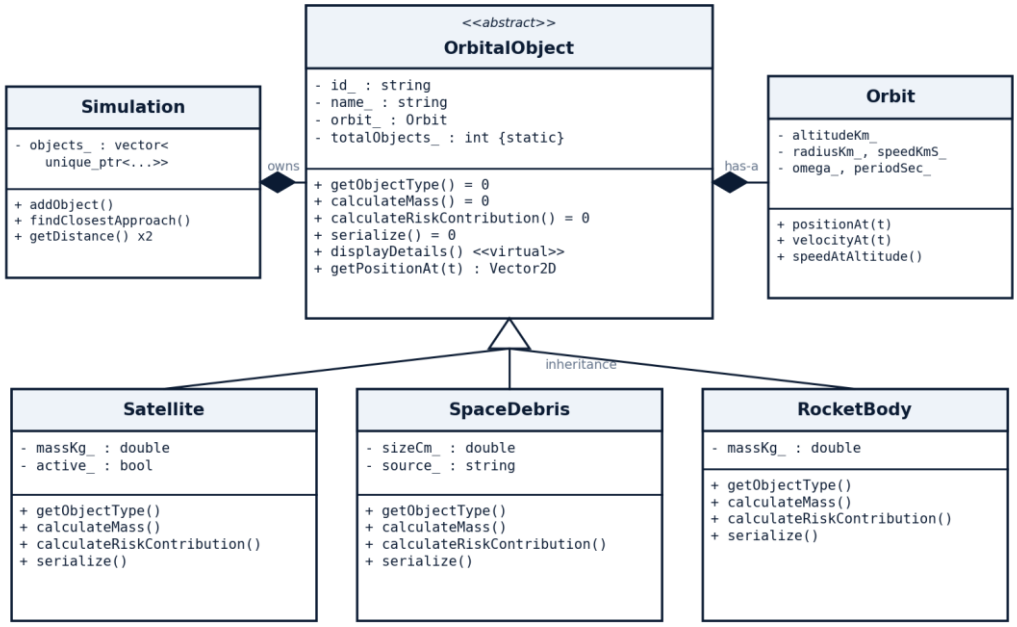


Figure 4.2: UML-style class diagram of the object hierarchy

4.3 Relationships Between Classes

Relationship	Classes	Explanation
Inheritance (is-a)	Satellite, SpaceDebris, RocketBody → OrbitalObject	Each child is a kind of OrbitalObject and reuses its ID, name and orbit.
Composition (has-a)	OrbitalObject → Orbit	An Orbit exists only as part of its object and is created and destroyed with it.
Composition (owns)	Simulation → OrbitalObject	Simulation stores `unique_ptr<OrbitalObject>` and deletes the objects when it is cleared or destroyed.
Composition	ConsoleUI → Simulation, RiskAnalyzer, DataManager, ReportGenerator	The menu class holds one of each manager.
Association	Conjunction ↔ objects	A Conjunction refers to its two objects by their IDs.
Dependency (uses)	RiskAnalyzer → Simulation, Conjunction, OrbitalObject; ReportGenerator → Conjunction, Simulation; DataManager → Simulation and the three concrete classes	These classes use others temporarily, for example as function parameters.

4.4 Project Workflow

Figure 4.3 shows the complete flow of data through SCOPE, from user input to saved results.

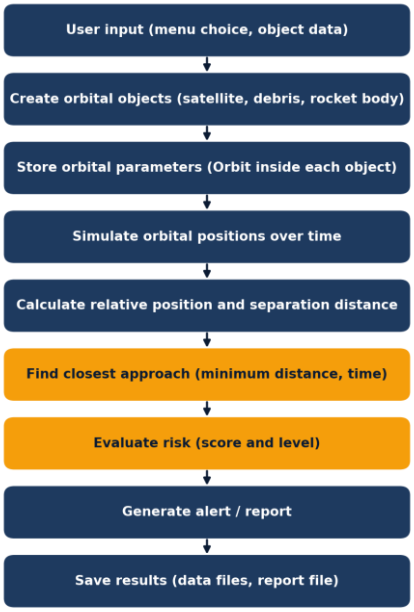


Figure 4.3: Project workflow

Stage	What happens	Class / function
1. User input	The user chooses a menu option and enters data; every value is validated.	ConsoleUI, InputHelper
2. Create objects	A Satellite, SpaceDebris or RocketBody is constructed with its Orbit.	Satellite, SpaceDebris, RocketBody
3. Store parameters	The object is added to the simulation after checking for a duplicate ID.	Simulation::addObject
4. Simulate positions	The position at any time is calculated from the circular-orbit equations.	Orbit::positionAt
5. Separation distance	The distance between two objects at the same time is computed.	Simulation::getDistance
6. Closest approach	The time scan finds the minimum distance, its time and the relative velocity.	Simulation::findClosestApproach
7. Evaluate risk	Points are added and converted to a risk level.	RiskAnalyzer::assess
8. Alert / report	The result is displayed and a report is built.	Conjunction::display, ReportGenerator
9. Save results	Objects are saved to data files; the report is saved to the reports folder.	DataManager, ReportGenerator

4.5 Class Descriptions

Each class is described below: its purpose, data members, important member functions, constructor and destructor, relationships and the OOP concepts it demonstrates.

Table 4.1 summarises every class: its purpose and data members, its important member functions, and its constructor, relationships and the OOP concepts it demonstrates.

Class	Purpose and data members	Key member functions	Constructor, relationships, OOP concepts
Vector2D	2D vector for positions (km) and velocities (km/s). Data: `x_`, `y_`.	`magnitude`, `distanceTo`, `operator+ - *`, friend `operator<<`	Constructor with default arguments. Used by Orbit and Simulation. Encapsulation, operator overloading, friend function.
Orbit	One circular orbit. Data: altitude, start angle, direction (+1/−1), radius, speed, ω, period.	`positionAt`, `velocityAt`, `angleAt`, static `speedAtAltitude`	Constructor validates (160 to 40,000 km, direction ±1) and throws `InvalidOrbitException`. Part of every OrbitalObject (composition). Encapsulation, exceptions, static member.
OrbitalObject (abstract)	Common identity and interface. Data: `id_`, `name_`, `orbit_`, static `totalObjects_`.	Pure virtual: `getObjectType`, `calculateMass`, `calculateRiskContribution`, `serialize`. Virtual: `displayDetails`. Static: `getTotalObjects`.	Constructor validates ID and name; virtual destructor; copy operations deleted. Base of the three object classes. Abstraction, inheritance, polymorphism, static, composition.
Satellite	Active or inactive satellite. Data: mass, active flag.	Overrides all virtual functions; risk contribution 2.0 (active) or 5.0 (inactive).	Calls base constructor; checks mass > 0. Inheritance, overriding.
SpaceDebris	Uncontrolled fragment. Data: size (cm), source text.	Mass estimated from size (aluminium sphere); risk contribution 7.0.	Calls base constructor. Inheritance, polymorphism.
RocketBody	Spent rocket stage. Data: mass.	Overrides all virtual functions; risk contribution 5.0.	Calls base constructor. Inheritance, polymorphism.
Conjunction	Result for one pair: IDs, minimum distance, time, relative velocity, scores, `RiskLevel`.	Getters, `isCloseApproach`, `isImminent`, `setRisk`, `display`, `operator<`	Created by Simulation, completed by RiskAnalyzer, read by ReportGenerator. Encapsulation, operator overloading, enum class.
Simulation	Owns the objects and runs the calculations. Data: `vector<unique_ptr<OrbitalObject>>`, duration, step.	`addObject`, `getObject`, two `getDistance` overloads, `findClosestApproach`, `findAllApproaches`	Default settings 120 min and 1 s; `unique_ptr` frees objects automatically. Polymorphic container, overloading, STL, exceptions.
RiskAnalyzer	Scores and classifies encounters. No data members (thresholds are in `Constants.h`).	`distancePoints`, `velocityPoints`, `levelFromScore`, `assess`, `analyzePair`, `analyzeAll`	Uses Simulation, Conjunction, OrbitalObject. Runtime polymorphism, `std::sort` with `operator<`.
DataManager	Saves and loads text files; builds sample data. Data: folder name.	`saveAll`, `loadAll`, static `populateSampleData`	Constructor takes the folder. File handling, exceptions, `std::map`, polymorphic `serialize()`.
ReportGenerator	Builds and saves the risk report. Data: folder name.	`buildReport`, `saveReport`	Constructor takes the folder. File handling, string streams.
ConsoleUI	Menu interface; coordinates only. Data: one each of Simulation, RiskAnalyzer, DataManager, ReportGenerator.	`run`, `showMenu`, `handleChoice`, one handler per menu item	Composition, central exception handling, template-based input.
Support headers	`Constants.h` (all fixed values), `Exceptions.h` (`ScopeException` and 7 child classes), `InputHelper.h` (input templates).	`readNumber<T>`, `readNumberInRange<T>`, `readLine`, `trim`	Templates, exception inheritance, no magic numbers.

5. OOP Concepts Used

Every concept below is used for a real purpose in SCOPE; none was added only for appearance. Concepts that do not fit the problem, such as multiple inheritance, were deliberately left out.

Concept	Where it is used	Why it is useful
Classes and objects	All 12 classes	Each real-world idea is one class.
Encapsulation	All data members are private; `Orbit` validates its input	An object cannot be put into an invalid state.
Constructors	`Orbit(altitude, angle, direction)`; derived constructors call the base constructor	Objects are always created valid.
Destructor	`virtual ~OrbitalObject()`	Correct clean-up through base pointers; updates the object counter.
Inheritance	Satellite, SpaceDebris, RocketBody from OrbitalObject	Common code is written once.
Abstraction	`OrbitalObject` has pure virtual functions	Defines what an object must do without saying how.
Runtime polymorphism	Virtual calls through base references and pointers	Same code works for every object type.
Function overloading	`Simulation::getDistance(obj, obj, t)` and `getDistance(id, id, t)`	One task, two convenient forms.
Operator overloading	`Vector2D + - *` and `<<`; `Conjunction::operator<`	Readable vector maths; sorting by severity.
Static members	`OrbitalObject::totalObjects_`, `Orbit::speedAtAltitude`	Class-level data and utilities.
Friend function	`operator<<` for `Vector2D`	Standard way to print an object with `cout`.
Templates	`InputHelper::readNumber<T>`, `readNumberInRange<T>`	One routine reads `int` or `double` safely.
STL containers	`vector<unique_ptr<OrbitalObject>>`, `vector<Conjunction>`, `map<string, vector<string>>`	Dynamic storage, sorting, grouping.
Composition	OrbitalObject has an Orbit; Simulation owns objects; ConsoleUI owns managers	Lifetimes are managed automatically.
Exception handling	`Exceptions.h`; caught in `ConsoleUI::run`	The program never crashes on user mistakes.
File handling	`DataManager`, `ReportGenerator`	Data and reports persist.

5.1 Where Polymorphism Occurs

All objects are stored in one container of type `vector<unique_ptr<OrbitalObject>>`. The program never needs to know whether an element is a Satellite, SpaceDebris or RocketBody; it simply calls the virtual function and the correct version runs. The main places are:

- `Simulation::listObjects` calls `getObjectType()` and `calculateMass()` for each object.
- `RiskAnalyzer::assess` calls `calculateRiskContribution()` on both objects of a pair (2.0, 5.0 or 7.0 depending on the real type).
- `DataManager::saveAll` calls `getObjectType()` to choose the file and `serialize()` to create each line.
- `ConsoleUI::viewObjects` calls `displayDetails()`, which prints different extra information for each type.

Benefit: a new object type, for example a `Probe` class, only needs to derive from `OrbitalObject` and implement the five virtual functions. `Simulation` and `RiskAnalyzer` need no change.

5.2 Abstraction and Encapsulation

`OrbitalObject` is abstract because it contains pure virtual functions; trying to create one directly gives a compile error. This matches reality: there is no "generic" orbital object, only satellites, debris and rocket bodies. Encapsulation protects the data: for example, the only way to create an `Orbit` is through its constructor, which rejects an altitude outside 160 to 40,000 km.

6. Mathematical Model

SCOPE uses a deliberately simple model so that the programming, not the mathematics, is the main subject. All distances are in kilometres and time is in seconds.

Quantity	Formula	Meaning
Orbital radius	$r = R + h$	$R = 6371$ km, h = altitude
Circular orbital speed	$v = \sqrt{(GM / r)}$	$GM = 398600.4418$ km³/s²
Angular velocity	$\omega = v / r$	Radians per second
Orbital period	$T = 2\pi / \omega$	Seconds per revolution
Angle at time t	$\theta(t) = \theta_0 \pm \omega \cdot t$	+ prograde, − retrograde
Position	$x = r \cos\theta$, $y = r \sin\theta$	Point on a circle around Earth
Velocity	$\vec{v} = \pm v\,(-\sin\theta, \cos\theta)$	Tangent to the circle
Relative position	$\Delta = P_B - P_A$	Vector from object A to object B
Separation distance	$d = \sqrt{(\Delta x^2 + \Delta y^2)}$	Length of the relative position
Relative velocity	$\lvert\vec{v}_A - \vec{v}_B\rvert$	Closing speed at a given time
Debris mass (estimate)	$m = \rho \cdot (4/3) \cdot \pi \cdot (D/2)^3$	$\rho = 2700$ kg/m³ (aluminium), D = diameter

6.1 Worked Example

For an object at altitude $h = 550$ km:

- Orbital radius: $r = 6371 + 550 = 6921$ km.
- Speed: $v = \sqrt{(398600.4418 / 6921)} = 7.59$ km/s.

- Angular velocity: $\omega = 7.59 / 6921 = 0.0010965$ rad/s.
- Period: $T = 2\pi / \omega = 5730$ s = 95.5 minutes.

These values match the program output for SAT-001 shown in Chapter 9 (altitude 550.00 km, radius 6921.00 km, speed 7.59 km/s, period 95.50 min).

6.2 Why Close Approaches Are Possible

All objects lie in one plane and move on circles. Two objects at different altitudes that move in the **same** direction slowly drift apart and never come close. Two objects that move in **opposite** directions cross each other every half of their relative revolution. When their angles are equal, their separation is only the difference between their orbital radii, so the minimum distance is approximately $|r_1 - r_2|$. For SAT-001 (550 km) and DEB-001 (551.82 km) this gives 1.82 km, which is exactly the result produced by the program. At that moment the relative speed is about $v_1 + v_2 \approx 15$ km/s. This is why the sample debris objects were placed in retrograde orbits.

6.3 Why the Model Is Simplified

A two-dimensional circle needs only trigonometry and keeps the code short and understandable. Real orbits differ in important ways:

- They are three-dimensional ellipses with inclination, not circles in one plane.
- Earth's shape, atmospheric drag, the Sun and the Moon, and solar radiation pressure change the orbits over time.
- Positions are not known exactly; they come from measurements with uncertainty.
- Real conjunction assessment calculates a probability of collision from this uncertainty and from the sizes of the objects.

7. Algorithms

7.1 Closest-Approach Detection

For every pair of objects the program searches the simulated time window (default 120 minutes, 1-second step) for the smallest separation.

1. Compute the position of both objects at $t = 0$ and at every time step $t = \Delta t, 2\Delta t, \ldots$ up to the duration.
2. At each time calculate the separation $d(t) = |P_B(t) - P_A(t)|$.
3. A sample that is not larger than both of its neighbours is a **local minimum**: the objects were closest around that time.
4. Around each local minimum, shrink the time interval repeatedly (60 passes of a ternary search) to find the true minimum between two samples. This is necessary because at about 15 km/s the objects move about 15 km between samples.
5. Keep the smallest refined minimum. If two minima are equal within 10^{-6} km the earlier one is kept.
6. Calculate the relative velocity $|\vec{v}_A - \vec{v}_B|$ at the time of the closest approach.

7. If the minimum distance is at most 25 km, flag the pair as a **close approach**.

Cost: with N objects there are N(N − 1)/2 pairs and about duration/step samples per pair. For the sample data this is 21 pairs × 7200 samples ≈ 151,000 distance calculations, which run instantly.

7.2 Risk Assessment Model

The model is transparent: the final score is the sum of three parts, each read from a small table. If the pair never comes within 25 km, the risk is LOW and no points are added.

Factor	Rule	Points
Distance (maximum 60)	≤ 1 km ≤ 5 km ≤ 10 km ≤ 25 km	60 45 30 15
Relative velocity (maximum 25)	≥ 10 km/s ≥ 5 km/s ≥ 1 km/s	25 15 8
Object type (each object)	Space debris Rocket body Inactive satellite Active satellite	7 5 5 2

Total score	Risk level
80 or more	CRITICAL
55 to 79	HIGH
30 to 54	MODERATE
Below 30 (or no close approach)	LOW

Example: SAT-001 and DEB-001 have a minimum distance of 1.82 km (45 points), a relative velocity of 15.18 km/s (25 points) and the object types active satellite (2) + debris (7) = 9 points. The total is 45 + 25 + 9 = 79, which is **HIGH**.

Important: these thresholds were chosen by the author for teaching. They are **not** official ISRO, NASA, ESA or international collision-risk standards. The time of closest approach is reported (and marked “imminent” if it is within 30 minutes) but does not change the score.

8. Implementation

This chapter shows the most important parts of the source code with short explanations. The complete code is in the project folder.

8.1 The Orbit Class

The constructor validates its arguments first and then computes the derived quantities once. Throwing from a constructor means an invalid `Orbit` can never exist.

Listing 8.1: Orbit constructor (src/Orbit.cpp)

```
Orbit::Orbit(double altitudeKm, double startAngleDeg, int direction)
    : altitudeKm_(altitudeKm), startAngleDeg_(startAngleDeg), direction_(direction) {
    // Validate BEFORE doing any maths; an object must never exist in an invalid state.
    if (!(altitudeKm >= Constants::MIN_ALTITUDE_KM && altitudeKm <=
Constants::MAX_ALTITUDE_KM)) {
        throw InvalidOrbitException("altitude must be between " +

std::to_string(static_cast<int>(Constants::MIN_ALTITUDE_KM)) + " and " +

std::to_string(static_cast<int>(Constants::MAX_ALTITUDE_KM)) + " km.");
    }
    if (direction != 1 && direction != -1) {
        throw InvalidOrbitException("direction must be +1 (prograde) or -1 (retrograde).");
    }
    radiusKm_  = Constants::EARTH_RADIUS_KM + altitudeKm_;            // r = R_E + h
    speedKmS   = std::sqrt(Constants::EARTH MU KM3 S2 / radiusKm );   // v = sqrt(GM / r)
    omega_     = speedKmS_ / radiusKm_;                               // w = v / r
    periodSec_ = 2.0 * Constants::PI / omega_;                        // T = 2*pi / w
}
```

8.2 The Abstract Base Class

`OrbitalObject` declares the interface. The functions ending in `= 0` are pure virtual, so every concrete class must implement them. The destructor is virtual and the copy operations are deleted.

Listing 8.2: OrbitalObject class (include/OrbitalObject.h)

```
class OrbitalObject {
public:
    OrbitalObject(const std::string& id, const std::string& name, const Orbit& orbit);
    virtual ~OrbitalObject();                        // virtual: safe delete through base
pointer

    // Objects have a unique identity, so copying is disabled.
    OrbitalObject(const OrbitalObject&) = delete;
    OrbitalObject& operator=(const OrbitalObject&) = delete;

    // ---- Common (non-virtual) behaviour ----
    const std::string& getId() const;
    const std::string& getName() const;
    const Orbit&       getOrbit() const;
    Vector2D getPositionAt(double timeSec) const;
    Vector2D getVelocityAt(double timeSec) const;

    // ---- Polymorphic behaviour: every derived class MUST/CAN provide its own ----
    virtual std::string getObjectType() const = 0;        // "Satellite", "Space
Debris", ...
    virtual double calculateMass() const = 0;             // kg
    virtual double calculateRiskContribution() const = 0; // points used by RiskAnalyzer
    virtual std::string serialize() const = 0;            // one text line for the data
file
    virtual void displayDetails(std::ostream& os = std::cout) const; // base prints common
part

    static int getTotalObjects();                         // how many objects exist right
now

protected:
    std::string serializeCommon() const;                  //
id|name|altitude|angle|direction

private:
    std::string id ;
    std::string name_;
    Orbit       orbit_;      // composition: an object HAS an orbit
    static int  totalObjects_;
};
```

8.3 Derived Classes

Each derived class overrides the virtual functions in its own way. Debris estimates its mass from its size and has the highest risk contribution; a satellite's contribution depends on whether it can manoeuvre.

Listing 8.3: SpaceDebris mass and risk (src/SpaceDebris.cpp)

```
// Mass of a solid aluminium sphere: m = density * (4/3) * pi * radius^3  (a rough
estimate)
double SpaceDebris::calculateMass() const {
    double radiusM = (sizeCm_ / 100.0) / 2.0;
    return Constants::ALUMINIUM DENSITY KG M3 * (4.0 / 3.0) * Constants::PI * radiusM *
radiusM * radiusM;
}

// Debris is uncontrolled and cannot avoid anything -> highest contribution.
double SpaceDebris::calculateRiskContribution() const { return 7.0; }
```

8.4 Operator Overloading

`Vector2D` overloads `+`, `-` and `*` and the stream operator `<<` (see `src/Vector2D.cpp`).

With these operators, the separation of two objects is written as `(posB - posA).magnitude()`, which reads like the mathematical formula.

8.5 Closest-Approach Search

Listing 8.4: Simulation::findClosestApproach (src/Simulation.cpp)

```
// Closest-approach search (two phases):
//  Phase 1 (coarse scan): sample the separation every 'stepSec_' seconds and note every
local
//                         minimum (a sample smaller than both neighbours = the pair got
closest here).
//  Phase 2 (refine)     : around each local minimum, repeatedly shrink the time interval
//                         (ternary search) to pinpoint the true minimum between two
samples.
// The smallest refined minimum is the closest approach; on ties the EARLIEST one is kept.
Conjunction Simulation::findClosestApproach(const OrbitalObject& a, const OrbitalObject& b)
const {
    const long coarseSteps = static_cast<long>(durationSec_ / stepSec_);
    double bestTime = 0.0;
    double bestDist = std::numeric_limits<double>::max();

    auto refine = [&](double centre) {
        double lo = std::max(0.0, centre - stepSec_);
        double hi = std::min(durationSec_, centre + stepSec_);
        for (int k = 0; k < Constants::REFINE_ITERATIONS; ++k) {
            double m1 = lo + (hi - lo) / 3.0;
            double m2 = hi - (hi - lo) / 3.0;
            if (getDistance(a, b, m1) < getDistance(a, b, m2)) hi = m2; else lo = m1;
        }
        double t = (lo + hi) / 2.0;
        double d = getDistance(a, b, t);
        if (d < bestDist - Constants::TIE_TOLERANCE_KM) { bestDist = d; bestTime = t; }
    };

    double previous = getDistance(a, b, 0.0);
    double current  = coarseSteps >= 1 ? getDistance(a, b, stepSec ) : previous;
    if (current >= previous) refine(0.0);                        // minimum at the very
start
    for (long i = 1; i < coarseSteps; ++i) {
        double next = getDistance(a, b, (i + 1) * stepSec_);
        if (current <= previous && current <= next) refine(i * stepSec_);   // local
minimum
        previous = current;
        current  = next;
    }
    if (coarseSteps >= 1 && current < previous) refine(coarseSteps * stepSec_); // minimum
at the end
```

```
    // Relative velocity = length of (velocity A - velocity B) at that moment.
    double relVel = (a.getVelocityAt(bestTime) - b.getVelocityAt(bestTime)).magnitude();
    return Conjunction(a.getId(), b.getId(), a.getObjectType(), b.getObjectType(),
                       bestDist, bestTime, relVel);
}
```

The lambda `refine` performs the interval-shrinking search. The loop checks each sample against its two neighbours (`previous`, `current`, `next`) to find local minima.

8.6 Risk Assessment and Polymorphism

Listing 8.5: RiskAnalyzer::assess (src/RiskAnalyzer.cpp)

```
void RiskAnalyzer::assess(Conjunction& c, const OrbitalObject& a, const OrbitalObject& b)
const {
    if (!c.isCloseApproach()) {            // far apart -> nothing to evaluate
        c.setRisk(RiskLevel::LOW, 0.0, 0.0, 0.0);
        return;
    }
    double distScore = distancePoints(c.getMinDistanceKm());
    double velScore  = velocityPoints(c.getRelVelocityKmS());
    // Polymorphism: each object reports its own contribution through the base reference.
    double objScore  = a.calculateRiskContribution() + b.calculateRiskContribution();
    c.setRisk(levelFromScore(distScore + velScore + objScore), distScore, velScore,
objScore);
}
```

The two calls to `calculateRiskContribution()` are made through `const OrbitalObject&` references. The program decides at run time which version to call. This is runtime polymorphism.

8.7 File Handling

`saveAll` groups the serialised lines by object type in a `std::map`, then writes one file per type. Each object produces its own line through the virtual function `serialize()`.

Listing 8.6: DataManager::saveAll (src/DataManager.cpp)

```
void DataManager::saveAll(const Simulation& sim) const {
    std::error_code ec;
    fs::create_directories(dataDir_, ec);
    if (ec) throw FileException("cannot create folder '" + dataDir  + "'.");


    // map: object type -> list of serialized lines (STL container)
    std::map<std::string, std::vector<std::string>> byType;
    for (const auto& obj : sim.getObjects())
        byType[obj->getObjectType()].push_back(obj->serialize());   // polymorphic
serialize()


    writeFile(Constants::SATELLITE_FILE,
"ID|Name|Altitude_km|StartAngle_deg|Direction|Mass_kg|Active", byType["Satellite"]);
    writeFile(Constants::DEBRIS_FILE,
"ID|Name|Altitude_km|StartAngle_deg|Direction|Size_cm|Source", byType["Space Debris"]);
    writeFile(Constants::ROCKET_FILE,
"ID|Name|Altitude km|StartAngle deg|Direction|Mass kg", byType["Rocket Body"]);
}
```

`loadSection` reads a file line by line, ignores blank lines and comments, rebuilds each object and skips corrupt lines instead of stopping.

Each file is a plain text file with fields separated by `|`. Comment lines begin with `#`:

Data file: data/satellites.txt

```
# ID|Name|Altitude_km|StartAngle_deg|Direction|Mass_kg|Active
# SIMULATED / EDUCATIONAL DATA - not real tracking data
SAT-001|EduSat-1|550|0|1|250|1
SAT-002|EduSat-2|700|90|1|420|1
SAT-003|WeatherSim-3|800|200|1|1200|1
```

8.8 Exception Handling

All SCOPE errors derive from one base class, so one `catch` block can handle them. Low-level code throws; the menu loop catches and prints a message.

Listing 8.7: ConsoleUI::run (src/ConsoleUI.cpp)

```
void ConsoleUI::run() {
    showBanner();
    bool running = true;
    while (running) {
        try {
            showMenu();
            int choice = readNumberInRange<int>("Enter your choice: ", 0, 12);
            running = handleChoice(choice);
        } catch (const InputClosedException&) {
            std::cout << "\nInput closed. Exiting SCOPE.\n";
            return;
        } catch (const ScopeException& e) {      // every expected error ends up here
            std::cout << "\n[ERROR] " << e.what() << "\n";
        } catch (const std::exception& e) {      // anything unexpected: still no crash
            std::cout << "\n[UNEXPECTED ERROR] " << e.what() << "\n";
        }
    }
    std::cout << "\nThank you for using SCOPE. Goodbye!\n";
}
```

8.9 Template for Safe Input

One function template reads any numeric type. It keeps asking until the whole line is a valid number, so letters or empty input never reach the calculations.

Listing 8.8: InputHelper::readNumber<T> (include/InputHelper.h)

```
// TEMPLATE: works for int, double, long ... Repeats until the whole line is a valid
number.
template <typename T>
T readNumber(const std::string& prompt,
             std::istream& in = std::cin, std::ostream& out = std::cout) {
    while (true) {
        std::string line = readLine(prompt, in, out);
        std::istringstream parser(line);
        T value;
        if (parser >> value && (parser >> std::ws).eof()) return value;
        out << "  Invalid number. Please enter a numeric value.\n";
    }
}
```

9. User Interface and Sample Output

SCOPE has a numbered console menu. The outputs below are the real output of the program using the simulated sample data (user input appears after each prompt).

9.1 Main Menu

Output 9.1: Start-up screen and menu

```
============================================
                  SCOPE
      SPACE COLLISION & ORBITAL
     PROXIMITY EVALUATION SYSTEM
============================================
  Educational simulation - NOT an operational
  collision-prediction system.

--------------------------------------------
  Objects loaded: 0
--------------------------------------------
  1. Add Satellite
  2. Add Space Debris
  3. Add Rocket Body
  4. View Orbital Objects
  5. Simulate Orbits
  6. Detect Close Approaches
  7. Analyze Collision Risk
  8. Generate Risk Report
  9. Save Data
 10. Load Data
 11. Load Sample Data (SIMULATED)
 12. Simulation Settings
  0. Exit
Enter your choice:
```

9.2 Loading Sample Data and Viewing Objects

Output 9.2: Options 11 and 4 (sample data, object table and details)

```
Enter your choice: 11

SIMULATED sample data loaded (7 objects).
Current objects were replaced.

Enter your choice: 4

--- Registered Orbital Objects (7) ---
ID        Type            Name                   Alt (km)   Mass (kg)
------------------------------------------------------------------------
SAT-001   Satellite       EduSat-1                 550.00      250.00
SAT-002   Satellite       EduSat-2                 700.00      420.00
SAT-003   Satellite       WeatherSim-3             800.00     1200.00
DEB-001   Space Debris    Fragment-A               551.82        2.44
DEB-002   Space Debris    Fragment-B               700.60        0.72
DEB-003   Space Debris    Fragment-C               812.00       11.31
RB-001    Rocket Body     SimStage-1               400.00     2100.00

Enter an ID for full details (or press Enter to skip): SAT-001

  Object ID       : SAT-001
  Name            : EduSat-1
  Type            : Satellite
  Altitude        : 550.00 km
  Orbital radius  : 6921.00 km
  Orbital speed   : 7.59 km/s
  Orbital period  : 95.50 min
  Direction       : Prograde
  Start angle     : 0.00 deg
  Mass            : 250.00 kg
  Status          : Active (can manoeuvre)
```

9.3 Simulating Orbits

Output 9.3: Option 5 (positions at 0 and 10 minutes)

```
Enter your choice: 5

--- Simulate Orbits ---
Start time (minutes): 0
Interval between snapshots (minutes): 10
Number of snapshots (1-10): 2

  Positions at t = 0.0 min
  ID                x (km)        y (km)   angle (deg)
  SAT-001          6921.00          0.00          0.00
  SAT-002             0.00       7071.00         90.00
  SAT-003         -6738.54      -2452.63        200.00
  DEB-001         -6855.45       -963.47        188.00
  DEB-002         -6964.17      -1227.97        190.00
  DEB-003          6220.66       3591.50         30.00
  RB-001           4787.82       4787.82         45.00

  Positions at t = 10.0 min
  ID                x (km)        y (km)   angle (deg)
  SAT-001          5476.39       4231.95         37.70
  SAT-002         -4206.23       5683.90        126.50
  SAT-003         -4036.75      -5926.88        235.74
  DEB-001         -6014.54       3427.94        150.32
  DEB-002         -6328.74       3155.09        153.50
  DEB-003          7148.08       -707.43        354.35
  RB-001            713.05       6733.35         83.96
```

9.4 Detecting Close Approaches

Output 9.4: Option 6 (close approaches below 25 km)

```
Enter your choice: 6

Scanning every pair for 120 min (step 1 s)...

*** CLOSE APPROACH DETECTED ***
  Object 1          : SAT-002 (Satellite)
  Object 2          : DEB-002 (Space Debris)
  Minimum Distance  : 0.60 km
  Time of Approach  : 13.70 minutes  (imminent)
  Relative Velocity : 15.02 km/s
  Status            : CRITICAL RISK

*** CLOSE APPROACH DETECTED ***
  Object 1          : SAT-001 (Satellite)
  Object 2          : DEB-001 (Space Debris)
  Minimum Distance  : 1.82 km
  Time of Approach  : 24.94 minutes  (imminent)
  Relative Velocity : 15.18 km/s
  Status            : HIGH RISK

*** CLOSE APPROACH DETECTED ***
  Object 1          : SAT-003 (Satellite)
  Object 2          : DEB-003 (Space Debris)
  Minimum Distance  : 12.00 km
  Time of Approach  : 26.61 minutes  (imminent)
  Relative Velocity : 14.90 km/s
  Status            : MODERATE RISK

3 of 21 pairs came closer than 25.00 km.
Closest pair overall: SAT-002 & DEB-002 at 0.60 km.
```

9.5 Analysing Risk

Output 9.5: Option 7 (all pairs, sorted from most to least severe)

```
Enter your choice: 7

--- Analyze Collision Risk ---
  1) One pair
  2) All pairs
Choice: 2

Pair                  Min (km)   TCA (min)       Rel v   Score  Risk
-----------------------------------------------------------------------
SAT-002/DEB-002           0.60       13.70       15.02      94  CRITICAL
SAT-001/DEB-001           1.82       24.94       15.18      79  HIGH
SAT-003/DEB-003          12.00       26.61       14.90      49  MODERATE
SAT-003/DEB-002          99.40       48.45       14.96       0  LOW
SAT-002/DEB-003         112.00       41.58       14.96       0  LOW
SAT-002/DEB-001         148.18       13.21       15.10       0  LOW
SAT-001/DEB-002         150.60       25.61       15.10       0  LOW
DEB-001/RB-001          151.82       18.66       15.26       0  LOW
SAT-003/DEB-001         248.18       47.40       15.04       0  LOW
SAT-001/DEB-003         262.00        4.09       15.04       0  LOW
DEB-001/DEB-002         285.97        0.00        0.28       0  LOW
DEB-002/RB-001          300.60       19.22       15.18       0  LOW
DEB-003/RB-001          412.00       46.24       15.12       0  LOW
SAT-002/RB-001         1898.22      120.00        2.06       0  LOW
SAT-001/RB-001         5241.53        0.00        5.84       0  LOW
SAT-001/SAT-002        8584.61      120.00        9.26       0  LOW
SAT-002/SAT-003       10979.34      120.00       11.54       0  LOW
SAT-003/RB-001        11853.51      120.00       12.86       0  LOW
DEB-001/DEB-003       12968.30      120.00       13.82       0  LOW
DEB-002/DEB-003       13764.11      120.00       14.44       0  LOW
SAT-001/SAT-003       13877.98        0.00       14.82       0  LOW
```

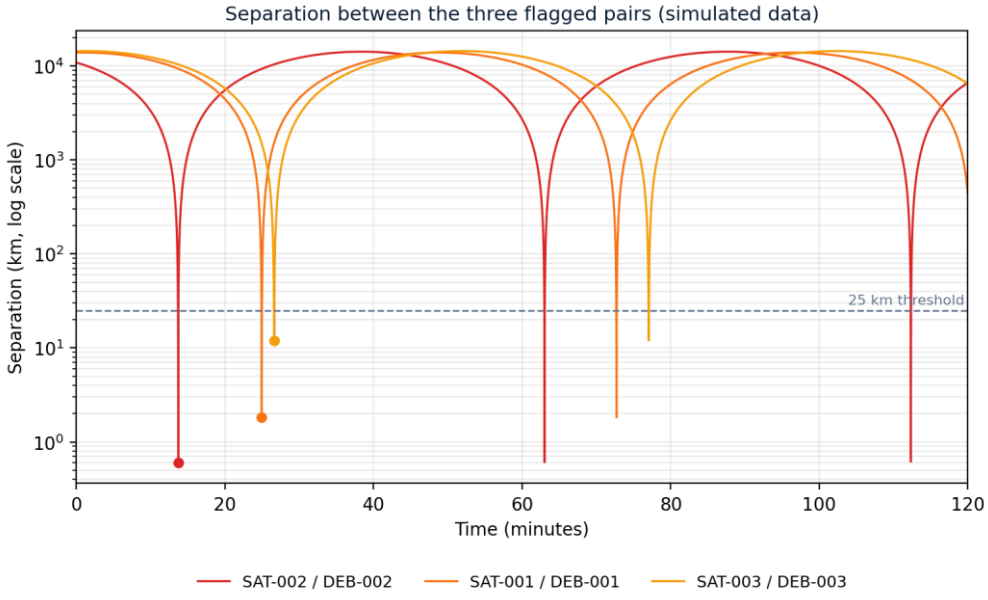


Figure 9.1: Separation between the three flagged pairs over 120 minutes (log scale)

The figure shows the separation of each flagged pair. The curve falls sharply when the objects pass each other; the dots mark the first closest approach reported by the program, and the dashed line is the 25 km close-approach threshold.

9.6 Risk Report

Output 9.6: Option 8 (start of the report; the full report is saved in the reports folder)

```
Enter your choice: 8


======================================================================
 SCOPE - COLLISION RISK REPORT
 Space Collision & Orbital Proximity Evaluation System
======================================================================
 Generated        : 2026-10-23 18:41:54
 DISCLAIMER       : EDUCATIONAL SIMULATION. Uses simulated data and a
                    simplified 2D circular-orbit model. Risk levels use
                    project-defined thresholds, NOT official standards.
 Objects analysed : 7
 Simulated window : 120 min, step 1 s
 Pairs analysed   : 21

 SUMMARY
   CRITICAL : 1
   HIGH     : 1
   MODERATE : 1
   LOW      : 18

 DETAILED RESULTS (most severe first)
----------------------------------------------------------------------
 #1  CLOSE APPROACH DETECTED
  Object 1          : SAT-002 (Satellite)
  Object 2          : DEB-002 (Space Debris)
  Minimum Distance  : 0.60 km
  Time of Approach  : 13.70 minutes  (imminent)
  Relative Velocity : 15.02 km/s
  Status            : CRITICAL RISK
  Score breakdown   : distance 60.00 + velocity 25.00 + object type 9.00 = 94.00 / 100
----------------------------------------------------------------------
```

9.7 Saving and Loading

Output 9.7: Option 9 (save) and option 10 (load)

```
Enter your choice: 9

Saved 7 objects to folder 'data'.

Enter your choice: 10

Loaded 7 objects (0 skipped).
```

9.8 Error Handling

Output 9.8: Invalid menu input and invalid altitude

```
Enter your choice: abc
  Invalid number. Please enter a numeric value.
Enter your choice: 99
  Value must be between 0 and 12.
Enter your choice: 1

--- Add Satellite ---
Object ID (e.g. SAT-004): SAT-004
Name: TestSat
Altitude above Earth (km): -100
Starting angle (degrees): 0
Direction  1) Prograde  2) Retrograde: 1

[ERROR] Invalid orbit: altitude must be between 160 and 40000 km.
```

Output 9.9: Load with no data files present

```
Enter your choice: 10

[ERROR] File error: no data files found in folder 'data'. Save data first.
```

10. Testing

Testing was done in two ways: automated tests in `tests/test_scope.cpp` (run with `make test`) and manual tests using the menu. The automated tests use only the standard library, with no external testing framework. The “Actual result” column below is the real output of the test program.

10.1 Automated Test Cases

ID	Test	Test input	Expected result	Actual result	Result
TC01	Add valid satellite	SAT-T1, 500 km, 0 deg, 100 kg	1 object stored, type Satellite	1 object, type Satellite	Pass
TC02	Add valid space debris	DEB-T1, 600 km, retrograde, 10 cm	stored; estimated mass about 1.41 kg	stored; mass 1.41 kg	Pass
TC03	Duplicate object ID	second object with ID SAT-T1	DuplicateIdException, list size stays 1	Object ID 'SAT-T1' already exists. (size 1)	Pass
TC04	Invalid (negative) altitude	altitude = -50 km	InvalidOrbitException	Invalid orbit: altitude must be between 160 and 40000 km.	Pass
TC05	Distance calculation	two 500 km objects, 90 deg apart, t = 0	r*sqrt(2) = 9717.06 km	9717.06 km	Pass
TC06	Orbital position	500 km, start 0 deg; t = 0 and t = T/4	t=0 -> (6871, 0); T/4 -> (0, 6871)	t=0 -> (6871, 0); T/4 -> (0.00, 6871.00)	Pass
TC07	Close-approach detection	SAT-001 vs DEB-001 (sample data), 120 min, 1 s step	min distance about 1.82 km near 25 min, flagged as close approach	1.82 km at 24.94 min, close=yes	Pass
TC08	Risk classification	four pairs from sample data	HIGH, CRITICAL, MODERATE, LOW	SAT-001/DEB-001=HIGH, SAT-002/DEB-002=CRITICAL, SAT-003/DEB-003=MODERATE, SAT-001/SAT-002=LOW	Pass
TC09	File saving	save the 7 sample objects to test_data_tmp/	three data files created and non-empty	satellites.txt, debris.txt, rocketbodies.txt created	Pass
TC10	File loading	load the files saved in TC09	7 loaded, 0 skipped, results identical to original	7 loaded, 0 skipped; SAT-001/DEB-001 min distance 1.82 km	Pass
TC11	Missing file	load from a folder that does not exist	FileException, program continues	File error: no data files found in folder 'folder_that_does_not_exist'. Save data first.	Pass
TC12	Invalid user input	menu input sequence: abc, -5, 99, 4	'abc' rejected as non-numeric, -5 and 99 rejected as out of range, 4 accepted	accepted value = 4 after 3 rejections	Pass
TC13	Corrupt file lines	satellites.txt with 1 good and 2 bad lines	good line loaded, bad lines skipped, no crash	5 loaded, 2 skipped	Pass
TC14	Runtime polymorphism	3 objects stored as OrbitalObject pointers	Satellite=2.0 Space Debris=7.0 Rocket Body=5.0	Satellite=2.0 Space Debris=7.0 Rocket Body=5.0	Pass

Test cases TC01 to TC12 cover the required cases: adding a valid satellite and debris, duplicate ID, invalid altitude, distance calculation, orbital position, close-approach detection, risk classification, file saving and loading, a missing file, and invalid input. TC13 checks that corrupt lines in a data file are skipped (in that test the two bad lines are skipped, and the 1 valid line plus the 4 valid objects in the other two files are loaded, giving 5), and TC14 checks runtime polymorphism.

10.2 Manual Tests

Action in the program	Expected	Result
Enter `abc` at the menu	Message “Invalid number” and the prompt repeats	Pass
Enter `99` at the menu	Message “Value must be between 0 and 12”	Pass
Add a satellite with an existing ID	`[ERROR] Object ID 'SAT-001' already exists.` and the menu returns	Pass
Add a satellite with altitude −100	`[ERROR] Invalid orbit: altitude must be between 160 and 40000 km.`	Pass
Choose option 4 with no objects	Message that no objects are registered	Pass
Choose option 7 or 8 with fewer than two objects	Error message, no crash	Pass
Choose option 10 with no data files	`[ERROR] File error: no data files found…`	Pass
Close the input (Ctrl+D / Ctrl+Z)	Program exits cleanly	Pass

11. Results and Discussion

With the simulated sample data (3 satellites, 3 debris objects and 1 rocket body), SCOPE analysed 21 pairs over a 120-minute window with a 1-second step. Three pairs came closer than 25 km:

Pair	Minimum distance	Time of approach	Relative velocity	Score	Risk level
SAT-002 / DEB-002	0.60 km	13.70 min	15.02 km/s	94	CRITICAL
SAT-001 / DEB-001	1.82 km	24.94 min	15.18 km/s	79	HIGH
SAT-003 / DEB-003	12.00 km	26.61 min	14.90 km/s	49	MODERATE
Other 18 pairs	above 25 km	–	–	0	LOW

Discussion. The three flagged pairs match the scenarios the sample data was designed to produce: one critical, one high and one moderate encounter, while the remaining pairs are safely separated. The minimum distances equal the differences in orbital radius (0.60, 1.82 and 12.00 km), confirming the geometry described in Section 6.2. The relative velocities of about 15 km/s are close to the sum of the two orbital speeds, as expected for objects moving in opposite directions. The object-type points raise the score of encounters that involve uncontrolled debris, which agrees with the idea that objects that cannot manoeuvre are more dangerous.

The program runs the complete analysis of all pairs in well under a second on a normal computer, and the refinement step ensures that results do not depend on the chosen time step.

12. Limitations

- The model is two-dimensional and circular; real orbits are three-dimensional ellipses.
- Perturbations such as atmospheric drag, Earth's shape and gravity from the Sun and Moon are ignored.
- The data is simulated; no real tracking data is used.
- Positions are exact; there is no measurement uncertainty and no probability of collision.
- Objects are points and never manoeuvre.
- Objects in the same plane and direction at different altitudes never approach closely, so realistic encounters require opposite directions.
- The risk thresholds are educational and project-defined, not official standards.
- The program is a console application without graphics.

13. Future Scope

- Extend the model to three dimensions with inclination and elliptical orbits (Kepler elements).
- Add perturbations such as atmospheric drag and the Earth's J2 effect.
- Read real two-line element (TLE) data and propagate it with the SGP4 model.
- Add uncertainty (covariance) and calculate a probability of collision.
- Suggest simple avoidance manoeuvres for active satellites.
- Add a graphical interface that draws the orbits and encounters.
- Use a unit-test framework such as GoogleTest and store data in JSON or a database.

Because of the object-oriented design, most of these changes would affect only a few classes. For example, 3D orbits would change `Orbit` and `Vector2D` (or a new `Vector3D`), while `Simulation`, `RiskAnalyzer` and the object hierarchy would stay almost unchanged.

14. Conclusion

SCOPE successfully applies Object Oriented Programming to a space-related problem. An abstract base class and three derived classes model the orbiting objects; runtime polymorphism lets the simulation, risk analyser and file manager treat them uniformly; and encapsulation, overloading, templates, STL containers, exceptions and file handling are each used where they have a real purpose. The program finds the closest approach of every pair, calculates the relative velocity, classifies the risk with a transparent model, saves data and reports, and handles invalid input without crashing. All 14 automated tests pass. Its main value is showing how a real-world-inspired problem can be broken into cooperating classes in a clean, tested and explainable C++ program.

15. References

1. Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
2. E. Balagurusamy, Object-Oriented Programming with C++, McGraw-Hill.
3. cppreference.com, C++ standard library documentation (https://en.cppreference.com).
4. H. D. Curtis, Orbital Mechanics for Engineering Students, Elsevier.
5. NASA Orbital Debris Program Office, public overview pages (background reading only).
6. ESA Space Debris Office, public overview pages (background reading only).
7. Project source code: https://github.com/<your-username>/SCOPE-Space-Collision-Orbital-Proximity-Evaluation

Appendix A: Compiling and Running

Requirement: g++ with C++17 (version 9 or newer). On Windows, install MSYS2 and the `mingw-w64-ucrt-x86_64-gcc` package and add `C:\msys64\ucrt64\bin` to the Path. Run the commands from inside the project folder so that the `data` and `reports` folders are found.

Build and run commands

```
# Linux, macOS or MSYS2 with make
make
./scope

# Without make
g++ -std=c++17 -Iinclude main.cpp src/*.cpp -o scope
./scope          (Windows: scope.exe)

# Automated tests
make test
```

Quick demonstration: choose 11 (load sample data), 4 (view objects), 5 (simulate orbits), 6 (detect close approaches), 7 (analyse risk), 8 (generate report) and 9 (save data).